

Introduction to Chevron's Maelstrom Developer Automation Framework

This README documents the custom Chevron GEOS and thirdPartyLibs (TPL) environment. The GEOS/TPL github repos are in **[GEOS](https://github.com/GEOS-DEV/GEOS)**. The **[official documentation site for the GEOS and TPL packages is here]** (<https://geosx-geosx.readthedocs-hosted.com/en/latest/>).

It may be better viewed using a text mark-up viewer, such as, the Preview feature of VSCode. Using a web browser to view this directly on Chevron's Microsoft ADO can at times produce visual effects that are hard to read.

NOTE: Chevron has already fully transitioned to Alma8, a Linux OS that corresponds to RHEL8. All recent and new installations of GEOS target Alma8. Older installations suitable for CentOS7/RHEL7 are still available but they are deprecated and we advise against anyone using them unless they have no access to an Alma8 Linux environment.

Please contact the HPC R&D and Innovation team for questions and requests for 1-on-1 assistance.

For any issues applicable to the HPC environment and its resources, contact the [Ressim Support](#), or the [Seis Supp](#) Teams channels

Where to find GEOS binaries at Chevron HPC

The GEOS binaries are installed in two Chevron locations and are frequently updated using the main (or upon request by members of the Chevron team) specialized branches for GEOS or TPL. These locations are

- the official "Chap" s/w directory at `/chap/geos` based on the the more stable GEOS/TPL sources, and
- the experimental/R&D directory at `/data/saet/software/x86_64/RHEL8/GEOS`.

The latter one receives updates more frequently and it includes several experimental versions based on different branches or on different specialized compilers and platforms. Older installations suitable for CentOS7/RHEL7 are still available at `/data/saet/software/x86_64/RHEL7/GEOS` but they are deprecated and we advise against anyone using them unless they have no access to an Alma8 Linux environment.

We are providing "initialization" scripts (`Init-XXX.rc`) that conveniently prepare your Linux command shell correctly before running a specific GEOS variation. These are, respectively, under

- `/chap/geos/.init`, and
- `/data/saet/software/.init`.

Use the following to get a list of "InitXXX.rc" scripts corresponding to installations of CPU GEOS, most recent ones first ('\$' is the prompt):

```
$ ls -lt /data/saet/software/.init/GEOS/*RHEL8*CPU*  
  
/data/saet/software/.init/GEOS/Init-x86_64-RHEL8-GEOS-0.2.0-ATS-CPU-OPTO3-
```

```
Hypre-GCC_13.2.0-ompi_hpcx-OMP-relwithdebinf.rc
/data/saet/software/.init/GEOS/Init-x86_64-RHEL8-GEOS-0.2.0-ATSTEST-update-
hypre-CPU-OPTO3-Hypre-GCC_13.2.0-ompi_hpcx-OMP-relwithdebinf.rc
...
```

For GPU GEOS use

```
$ ls -lt /data/saet/software/.init/GEOS/*RHEL8*GPU*

/data/saet/software/.init/GEOS/Init-x86_64-RHEL8-GEOS-0.2.0-ATS-GPU-Hypre-
GCC-CUDA_12.2-ompi_hpcx-OMP-relwithdebinf.rc
...
```

How-to-Run GEOS Quick-Guide

The GEOS s/w can be run at Chevron either

- interactively on a Linux blade (usually for small model test or runs), or
- using the SLURM batch system (suitable for larger model sizes).

Before running GEOS we should "source" one of the InitXXX.rc scripts to prepare our shell environment.

Interactive runs on Chevron's Linux Blades

Interactive runs take place on the Linux blades, that are available via **TGX** <https://hpc-leostream.azure.chevron.net/> and are meaningful with smaller models or when we want to obtain test results quickly.

To run CPU GEOS at the command line using N MPI ranks in a BASH shell environment, select one of the INIT-XXX.rc files as shown above and follow these steps:

```
cd /where/the/Maelstrom/model-xml/is
export INITRC="/data/saet/software/.init/GEOS/Init-x86_64-RHEL8-GEOS-
0.2.0-ATS-CPU-OPTO3-Hypre-GCC_13.2.0-ompi_hpcx-OMP-relwithdebinf.rc"

source $INITRC
mpirun -np N $GEOS_DIR/bin/geosx -i ./my-Maelstrom-model.xml [-x x -y y -
z z ] [Other GEOS parameters]
```

To run CPU GEOS at the command line using N MPI ranks with M OpenMP threads per rank, follow these steps:

```
cd /where/the/Maelstrom/model-xml/is
export INITRC="/data/saet/software/.init/GEOS/Init-x86_64-RHEL8-GEOS-
0.2.0-ATS-CPU-OPTO3-Hypre-GCC_13.2.0-ompi_hpcx-OMP-relwithdebinf.rc"
```

```
source $INITRC
export OMP_NUM_THREADS=M
mpirun -np N $GEOS_DIR/bin/geosx -i ./my-Maelstrom-model.xml [-x x -y y -
z z ] [Other GEOS parameters]
```

If your Linux shell is not BASH you can "jump" to BASH and proceed as above:

```
/bin/bash
cd /where/the/Maelstrom/model-xml/is
export INITRC="/data/saet/software/.init/GEOS/Init-x86_64-RHEL8-GEOS-
0.2.0-ATS-CPU-OPTO3-Hypre-GCC_13.2.0-mpi_hpcx-OMP-relwithdebinf.rc"

source $INITRC
mpirun -np N $GEOS_DIR/bin/geosx -i ./my-Maelstrom-model.xml [-x x -y y -
z z ] [Other GEOS parameters]
```

As we install new variations of GEOS and TPL, we also update the corresponding "Init-XXX.rc". Please check often.

Interactive runs of GEOS examples on Chevron's Linux Blades

GEOS brings in a large collection of example models. Documentation can be found on GEOS site (<https://geosx-geosx.readthedocs-hosted.com/en/latest/docs/sphinx/basicExamples/Index.html>). There are also more advanced examples available. We keep copies of these example models under

- /data/saet/software/x86_64/RHEL8/GEOS/inputFiles, and
- /chap/geos/inputFiles

To quickly test any particular example model, make a complete copy of the directory with the model in a location you have write access to and then run GEOS on that model following the guidelines in the documentation.

For instance, to run the "**co2Injection**" (<https://geosx-geosx.readthedocs-hosted.com/en/latest/docs/sphinx/basicExamples/co2Injection/Example.html>), assuming a BASH shell, proceed as follows.

```
cd /where/you/can/save/files
rsync -av /chap/geos/inputFiles/compositionalMultiphaseWell .
cd compositionalMultiphaseWell

export INITRC="/chap/geos/.init/Init-x86_64-RHEL8-GEOS-0.2.0-2024-06-29-CPU-OPTO3-Hypre-
GCC_13.2.0-mpi_hpcx-OMP-relwithdebinf.rc"
source $INITRC
mpirun -np 4 $GEOS_DIR/bin/geosx -i ./simpleCo2InjTutorial_smoke.xml -x 1 -y 1 -z 4
...
Num ranks: 4
Max threads: 2
Allocated      64.0 B to the HOST  : LvArray::Array, std::allocator >, 1, camp::int_seq,
```

```

int, LvArray::ChaiBuffer>
Allocated 64.0 B to the HOST : LvArray::Array, std::allocator >, 1, camp::int_seq,
int, LvArray::ChaiBuffer>
Allocated 64.0 B to the HOST : LvArray::Array, std::allocator >, 1, camp::int_seq,
int, LvArray::ChaiBuffer>
Allocated 64.0 B to the HOST : LvArray::Array, std::allocator >, 1, camp::int_seq,
int, LvArray::ChaiBuffer>
GEOS version: 0.2.0 (feature/paludettomagl/mgr-improvements, sha1: ea7cc5814)
- c++ compiler: gcc 13.2.0
- openmp version: 201511
- MPI version: Open MPI v4.1.7a1, package: Open MPI root@hpc-kernel-03 Distribution,
ident: 4.1.7a1, repo rev: v4.1.5-106-gefbeca7056, Unreleased developer copy
- HDF5 version: 1.12.1
- Conduit version: 0.8.2
- VTK version: 9.2.6
- RAJA version: 2023.6.1
- umpire version: 2023.6.0
- adiak version: ..
- caliper version: 2.10.0
- METIS version: 5.1.0
- PARAMETIS version: 4.0.0
- scotch version: 7.0.3
- superlu_dist version: 6.3.0
- suitesparse version: 5.7.9
- hypre version: v2.31.0-26-gf6cfb0355 (mgr-coarse-grid)
- Python3 version: 3.11.7
Started at 2024-07-03 16:20:52.204402254
Allocated 12.0 B to the HOST : LvArray::Array, int, LvArray::ChaiBuffer>
Allocated 12.0 B to the HOST : LvArray::Array, int, LvArray::ChaiBuffer>
Allocated 12.0 B to the HOST : LvArray::Array, int, LvArray::ChaiBuffer>

...
wellControls: BHP (at the specified reference elevation): 50000000 Pa
wellControls: Total rate: 0.007396443232569583 kg/s; total surface volumetric rate:
0.003958681478923127 sm3/s
wellControls: Phase 0 surface volumetric rate: 0.003958681478923127 sm3/s
wellControls: Phase 1 surface volumetric rate: 0 sm3/s
compositionalMultiphaseFlow: max relative pressure change during time step = 0.000 %
compositionalMultiphaseFlow: max absolute phase volume fraction change during time step =
0.008
coupledFlowAndWells: time-step required will be increased based on number of iterations.
Cleaning up events
Rank 0: Writing out restart file at
./simpleCo2InjTutorial_smoke_restart_000001014/rank_0000000.hdf5
Rank 2: Writing out restart file at
./simpleCo2InjTutorial_smoke_restart_000001014/rank_0000002.hdf5
Rank 3: Writing out restart file at
./simpleCo2InjTutorial_smoke_restart_000001014/rank_0000003.hdf5
Rank 1: Writing out restart file at
./simpleCo2InjTutorial_smoke_restart_000001014/rank_0000001.hdf5
coupledFlowAndWells, number of time steps: 1014

```

```

coupledFlowAndWells, number of successful nonlinear iterations: 1999
coupledFlowAndWells, number of successful linear iterations: 17572
coupledFlowAndWells, number of time step cuts: 0
coupledFlowAndWells, number of discarded nonlinear iterations: 0
coupledFlowAndWells, number of discarded linear iterations: 0
coupledFlowAndWells: apply solution time = 0.962277234 s (min), 1.268282894 s (max)
coupledFlowAndWells: assemble time = 12.184481448 s (min), 15.33624334 s (max)
coupledFlowAndWells: convergence check time = 0.139309894 s (min), 0.187321753 s (max)
coupledFlowAndWells: linear solver create time = 7.434506798 s (min), 7.474812272 s (max)
coupledFlowAndWells: linear solver setup time = 16.7292606 s (min), 18.488317352 s (max)
coupledFlowAndWells: linear solver solve time = 12.253901549 s (min), 14.022472297 s (max)
coupledFlowAndWells: linear solver total time = 38.791622069 s (min), 39.143023017 s (max)
coupledFlowAndWells: update state time = 2.671273046 s (min), 2.776576619 s (max)
Umpire                HOST sum across ranks:    46.3 MB
Umpire                HOST                rank max:    11.8 MB
Finished at 2024-07-03 16:22:08.508806264
total time              00h01m16s (76.30440401 s)
initialization time     00h00m02s (2.025110305 s)
run time                00h01m13s (73.820039489 s)

```

GEOS runs on Chevron's HPC batch clusters (Slurm)

Larger GEOS models should run on collections of powerful HPC nodes ("HPC clusters") under the control of a "Batch Job Scheduler". Chevron's HPC recently transitioned its batch systems to the Slurm scheduler and no further discussion will take place here on PBS batch.

GEOS should use the compute resources under Slurm "partitions" (batch queues "**geos**" or "**rso**"). These partitions have been properly configured to run the GEOS binaries. Other partitions, such as, geophysics's "**parallel**" may not, as of the time of this writing be ready to run GEOS. This however may be rectified soon so always check.

Slurm batch scripts

To facilitate running GEOS jobs under Slurm, we are providing helper Slurm scripts and script templates that are available at this [ADO site] (https://dev.azure.com/chevron/SubSurf-DIGITAL-GEOSX/_git/SubSurf-DIGITAL-GEOSX-scripts). Clone first this repos at your Linux account.

```

mkdir -p ${HOME}/myGEOS
cd ${HOME}/myGEOS
git clone https://chevron@dev.azure.com/chevron/SubSurf-DIGITAL-GEOSX/_git/SubSurf-DIGITAL-GEOSX-scripts
export SCR=${HOME}/myGEOS/SubSurf-DIGITAL-GEOSX-scripts

```

Slurm batch examples

In the following examples we show how to submit Slurm jobs to the "geos" Slurm partition. We will use GEOS binaries built with the OpenMPI stack.

To run GEOS on one Azure HPC node with SKU type "hmv3" and split the computation to **4** MPI ranks with one OpenMP thread per rank, do:

```
cd /where/my/Maelstrom/model-xml/is
export SCR=${HOME}/myGEOS/SubSurf-DIGITAL-GEOSX-scripts
export INITRC="/data/saet/software/.init/GEOS/Init-x86_64-RHEL8-GEOS-0.2.0-
ATS-CPU-OPTO3-Hypre-GCC_13.2.0-mpi_hpcx-OMP-relwithdebininfo.rc"

export OMP_NUM_THREADS=1
source $INITRC
NP=4 sbatch -N 1 -p geos --account=geos --constraint=hmv3
$SCR/GEOS_CPU_mpi.sh -i ./myModel.xml [-x x -y y -z z ] [Other GEOS
parameters]
```

To run GEOS on 4 Azure HPC node with SKU type "hmv3", get more verbose messages on the progress of the job, split the computation to **20** MPI ranks, with **5** ranks per node and with **2** OpenMP thread per rank, do:

```
cd /where/my/Maelstrom/model-xml/is
export SCR=${HOME}/myGEOS/SubSurf-DIGITAL-GEOSX-scripts
export INITRC="/data/saet/software/.init/GEOS/Init-x86_64-RHEL8-GEOS-0.2.0-
ATS-CPU-OPTO3-Hypre-GCC_13.2.0-mpi_hpcx-OMP-relwithdebininfo.rc"

export OMP_NUM_THREADS=2

NP=20 PPN=5 V=1 sbatch -N 4 -p geos --account=geos --constraint=hmv3
$SCR/GEOS_CPU_mpi.sh -i ./myModel.xml [-x x -y y -z z ] [Other GEOS
parameters]
```

There are many different ways we can set up the run-time configurations of GEOS. These are beyond the scope of this write-up and we advise users to contact the Innovation and HPC R&D crew for one on one assistance and guidance.

HPC apps are submitted to HPC clusters as "Slurm Batch Jobs" using annotations inside scripts that are interpreted by the SLURM system and direct it to allocate nodes and other resources to this application. The SLURM scheduler first allocates the resources that are indicated in the job script or at the command line, it launches the job and monitors it through its termination at which time it recovers the allocated resources and makes them available to other jobs. The [SLURM script annotations are described here]

([https://chevron.sharepoint.com/:w:/r/sites/HPCHome/Shared Documents/HPC Active Documents/SLURM/Slurm_User_Guide.docx?](https://chevron.sharepoint.com/:w:/r/sites/HPCHome/Shared Documents/HPC Active Documents/SLURM/Slurm_User_Guide.docx?d=w2659bec5d899473e80007fec2e442e0&csf=1&web=1&e=uQ54Fr)

[d=w2659bec5d899473e80007fec2e442e0&csf=1&web=1&e=uQ54Fr](https://chevron.sharepoint.com/:w:/r/sites/HPCHome/Shared Documents/HPC Active Documents/SLURM/Slurm_User_Guide.docx?d=w2659bec5d899473e80007fec2e442e0&csf=1&web=1&e=uQ54Fr)) Please note that any SLURM option that is specified within a script can be given as a command line option to the "sbatch" Slurm command that is used with job submissions.

Running co2Injection as a batch job under Slurm

Continuing with the same example model, to run the "**co2Injection**" (<https://geosx-geosx.readthedocs-hosted.com/en/latest/docs/sphinx/basicExamples/co2Injection/Example.html>), assuming a BASH shell, as a Slurm batch job, proceed as follows.

```
cd /where/you/can/save/files
rsync -av /chap/geos/inputFiles/compositionalMultiphaseWell .
cd compositionalMultiphaseWell

export INITRC="/chap/geos/.init/Init-x86_64-RHEL8-GEOS-0.2.0-2024-06-29-CPU-OPTO3-Hypre-
GCC_13.2.0-mpi_hpcx-OMP-relwithdebinfo.rc"

OMP_NUM_THREADS=2 NP=4 sbatch -p geos --account=geos --constraint=hbv3 --exclusive
$SCR/GEOS_CPU_mpi.sh -i ./simpleCo2InjTutorial_smoke.xml -x 1 -y 1 -z 4

Submitted batch job 1848086
...
```

After the job terminates, you can inspect the Slurm output using the Linux **less** or (**more**) commands, as follows

```
less slurm-1848086.out
... Lots of debug and status information ...
...
      ( Rflow ) = ( 6.41e-06 )      ( Rwell ) = ( 2.92e-12 )      ( R ) = ( 6.41e-
06 )
wellControls: BHP (at the specified reference elevation): 50000000 Pa
wellControls: Total rate: 0.007396444546595567 kg/s; total surface volumetric rate:
0.003958682182208498 sm3/s
wellControls: Phase 0 surface volumetric rate: 0.003958682182208498 sm3/s
wellControls: Phase 1 surface volumetric rate: 0 sm3/s
compositionalMultiphaseFlow: max relative pressure change during time step = 0.000 %
compositionalMultiphaseFlow: max absolute phase volume fraction change during time step =
0.008
coupledFlowAndWells: time-step required will be increased based on number of iterations.
Cleaning up events
Rank 2: Writing out restart file at
./simpleCo2InjTutorial_smoke_restart_000001014/rank_0000002.hdf5
Rank 0: Writing out restart file at
./simpleCo2InjTutorial_smoke_restart_000001014/rank_0000000.hdf5
Rank 3: Writing out restart file at
./simpleCo2InjTutorial_smoke_restart_000001014/rank_0000003.hdf5
Rank 1: Writing out restart file at
./simpleCo2InjTutorial_smoke_restart_000001014/rank_0000001.hdf5
coupledFlowAndWells, number of time steps: 1014
coupledFlowAndWells, number of successful nonlinear iterations: 1999
coupledFlowAndWells, number of successful linear iterations: 17568
```

```

coupledFlowAndWells, number of time step cuts: 0
coupledFlowAndWells, number of discarded nonlinear iterations: 0
coupledFlowAndWells, number of discarded linear iterations: 0
coupledFlowAndWells: apply solution time = 0.764882159 s (min), 1.34249192 s (max)
coupledFlowAndWells: assemble time = 9.512283213 s (min), 13.457403144 s (max)
coupledFlowAndWells: convergence check time = 0.115452504 s (min), 0.172757363 s (max)
coupledFlowAndWells: linear solver create time = 4.503109801 s (min), 4.536743149 s (max)
coupledFlowAndWells: linear solver setup time = 18.387750299 s (min), 19.561719854 s (max)
coupledFlowAndWells: linear solver solve time = 12.564405546 s (min), 13.740073964 s (max)
coupledFlowAndWells: linear solver total time = 36.986309459 s (min), 37.602123566 s (max)
coupledFlowAndWells: update state time = 1.842974732 s (min), 1.903651378 s (max)
Umpire                HOST sum across ranks:    46.3 MB
Umpire                HOST          rank max:    11.8 MB
Finished at 2024-07-03 16:43:49.450407235
total time             00h01m14s (74.948295008 s)
initialization time    00h00m03s (3.053776708 s)
run time               00h01m10s (70.783305403 s)

```

GEOS runs on Chevron's HPC batch clusters (Slurm)

Larger GEOS models should run on collections of powerful HPC nodes ("HPC clusters") under the control of a "Batch Job Scheduler". Chevron's HPC recently transitioned its batch systems to the Slurm scheduler and no further discussion will take place here on PBS batch.

GEOS should use the compute resources under Slurm "partition" "geos". This partition has been properly configured to run these binaries. The general "rso" or geophysics's "parallel" partitions may not be as of the time of this writing ready to run GEOS.

Slurm batch scripts

To facilitate running GEOS jobs under Slurm, we are providing helper Slurm scripts and script templates that are available at this [ADO site] (https://dev.azure.com/chevron/SubSurf-DIGITAL-GEOSX/_git/SubSurf-DIGITAL-GEOSX-scripts). Clone first this repos at your Linux account.

```

mkdir -p ${HOME}/myGEOS
cd ${HOME}/myGEOS
git clone https://chevron@dev.azure.com/chevron/SubSurf-DIGITAL-GEOSX/_git/SubSurf-DIGITAL-GEOSX-scripts
export SCR=${HOME}/myGEOS/SubSurf-DIGITAL-GEOSX-scripts

```

Slurm batch examples

In the following examples we show how to submit Slurm jobs to the "geos" Slurm partition. We will use GEOS binaries built with the OpenMPI stack.

To run GEOS on one Azure HPC node with SKU type "hmv3" and split the computation to **4** MPI ranks with one OpenMP thread per rank, do:


```
cd /where/my/Maelstrom/model-xml/is
export SCR=${HOME}/myGEOS/SubSurf-DIGITAL-GEOSX-scripts
export INITRC="/data/saet/software/.init/GEOS/Init-x86_64-RHEL8-GEOS-0.2.0-
ATS-CPU-OPTO3-Hypre-GCC_13.2.0-mpi_hpcx-OMP-relwithdebininfo.rc"

export OMP_NUM_THREADS=1
source $INITRC
NP=4 sbatch -N 1 -p geos --account=geos --constraint=hbv3
$SCR/GEOS_CPU_mpi.sh -i ./myModel.xml [-x x -y y -z z ] [Other GEOS
parameters]
```

To run GEOS on 4 Azure HPC node with SKU type "hbv3", get more verbose messages on the progress of the job, split the computation to **20** MPI ranks, with **5** ranks per node and with **2** OpenMP thread per rank, do:

```
cd /where/my/Maelstrom/model-xml/is
export SCR=${HOME}/myGEOS/SubSurf-DIGITAL-GEOSX-scripts
export INITRC="/data/saet/software/.init/GEOS/Init-x86_64-RHEL8-GEOS-0.2.0-
ATS-CPU-OPTO3-Hypre-GCC_13.2.0-mpi_hpcx-OMP-relwithdebininfo.rc"

export OMP_NUM_THREADS=2

NP=20 PPN=5 V=1 sbatch -N 4 -p geos --account=geos --constraint=hbv3
$SCR/GEOS_CPU_mpi.sh -i ./myModel.xml [-x x -y y -z z ] [Other GEOS
parameters]
```

There are many different ways we can set up the run-time configurations of GEOS. These are beyond the scope of this write-up and we advise users to contact the Innovation and HPC R&D crew for one on one assistance and guidance.

HPC apps are submitted to HPC clusters as "Slurm Batch Jobs" using annotations inside scripts that are interpreted by the SLURM system and direct it to allocate nodes and other resources to this application. The SLURM scheduler first allocates the resources that are indicated in the job script or at the command line, it launches the job and monitors it through its termination at which time it recovers the allocated resources and makes them available to other jobs. The [SLURM script annotations are described here]

([https://chevron.sharepoint.com/w:/r/sites/HPCHome/Shared Documents/HPC Active Documents/SLURM/Slurm_User_Guide.docx?](https://chevron.sharepoint.com/w:/r/sites/HPCHome/Shared Documents/HPC Active Documents/SLURM/Slurm_User_Guide.docx?d=w2659bec5d899473e80007fec2e442e0&csf=1&web=1&e=uQ54Fr)

[d=w2659bec5d899473e80007fec2e442e0&csf=1&web=1&e=uQ54Fr](https://chevron.sharepoint.com/w:/r/sites/HPCHome/Shared Documents/HPC Active Documents/SLURM/Slurm_User_Guide.docx?d=w2659bec5d899473e80007fec2e442e0&csf=1&web=1&e=uQ54Fr)) Please note that any SLURM option that is specified within a script can be given as a command line option to the "sbatch" Slurm command that is used with job submissions.

What is in this Chevron ADO site

This site contains Chevron-developed resources supporting Chevron's Maelstrom Exascale-Compute project. These resources enable the cloning, configuring, building and installing GEOS project packages under

different user configurations and build-time choices in a mostly automated fashion.

Chevron's framework can be used on any Azure Linux host, including Linux "blades" or HPC cluster nodes. Note that hosts that are not in Azure may run into GitHub access restrictions. We recommend using Linux blades that start with `lxspuscXXXX` as they reside in Azure and avoid using those starting with `lxsphouXXXX` that reside on-premises.

Chevron's GEOS framework provides scripts and other resources that support the entire GEOS workflow, including

- the ability to clone from Github of TPL and GEOS
- the ability to use custom Chevron `XXXX.cmake` files (under `GEOS/host-configs/CVX`) each enabling different options and MPI stacks to update and build GEOS/TPL for CPU-only and GPU-enabled environments
- allowing the users to specify options that build and install GEOS/TPL with different styles in user-selected locations,
- allowing the user to skip different steps, such as, updating sources from Github, going through the configuration process, and/or just build only specific targets, such as `geosx` the main GEOS binary, and
- implementing a "local caching" scheme that speeds up the very lengthy configure-build-install stages.
- The framework supports building and running GEOS MPI codes that are either CPU-only and GPU-enabled, using IntelMPI or OpenMPI (recommended) stacks.
- The framework also provides example SLURM scripts that allow performance scaling experiments of GEOS simulations in SLURM batch environments in a flexible manner.

The framework here automates the process of using Chevron or user provided CMAKE files to configure and build consistently

- thirdPartyLibs (TPL) and
- GEOS numerical package.

The build process allows GEOS to use different versions of Intel MPI and OpenMPI HPC-X. GEOS/TPL has been built and tested with

- GCC versions 10.2.0, 10.4.0 and 9.4.0
- Intel MPI versions 2021.9.0, 2021.5.1 and 2018.04, and
- HPC-X OpenMPI 4.1.x.

Note: We have recently encountered non-reproducible issues when running GEOS with the **IntelMPI** stack. **We strongly suggest users to build and run GEOS using the OpenMPI "HPC-X" MPI stack.**

We can choose to build GEOS/TPL with or without OpenMP enabled. We may also build GEOS/TPL with different compiler optimization levels. In large complex numerical packages like GEOS, very high optimization may result in higher round-off errors leading to lower precision that could impact the convergence of certain algorithms. We can select to build everything at the highest optimization level that does not impact numerical precision for our models.

GEOS/TPL may be built in "Release", "RelWitDebInfo" and "Debug" modes. The first two include all requested optimizations. "RelWitDebInfo" and "Debug" builds include symbolic debugging information with the binaries and libraries to enable debugging. "RelWitDebInfo" includes optimizations and includes debug information.

We have prepared a large number of CMAKE configurations each one requesting a particular MPI, OMP support and optimization level. These CMAKE files are Chevron specific and can be found under SubSurf-DIGITAL-GEOS-scripts/GEOS/host-configs/CVX with a composite name reflecting the specific settings.

All these s/w configurations are available and have been extensively tested to work properly in Chevron's HPC environment

Limitations

Currently, only the GNU compilers are supported. Other popular tool-chains, such as, the Intel (both "Classic" and OneAPI), Portland Group's (PGI) and Nvidia's HPC SDK compiles do not recognize certain C++ language extensions TPL packages use. We have successfully built and run GEOS/TPL with GCC version 11.4.0, 11.2.0 and 9.8.0. Newer versions and some not too old GNU versions should also work.

Getting Started Instructions

Usage Models

The framework supports two main GEOS usage models :

- **Model running:** users run different models using existing GEOS installations on the proper resources, and
- **GEOS/TPL building from sources:** SMEs update GEOS sources and run tests on models of interest.

Note : Below '\$' is the shell prompt and the discussion applies to **BASH** shells. We later provide additional instructions for using these environment from (the *highly discouraged*) [t]CSHell.

Cloning SubSurf-DIGITAL-GEOSX Repos from Chevron ADO

The very first task is to clone the ADO repos so we can use the various resources. We clone this on a Linux files system on which we have write access. Let's say that \$SCR environment variable points to this directory. We will keep using \$SCR to point to the location that SubSurf-DIGITAL-GEOSX-scripts has been cloned throughout this document.

```
$ cd /my/local/directory
$ export SCR=$(pwd)
$ git clone https://dev.azure.com/chevron/SubSurf-DIGITAL-GEOSX/_git/SubSurf-DIGITAL-GEOSX-scripts
```

Take a look at \$SCR/SubSurf-DIGITAL-GEOSX and familiarize yourself with its contents:

```
$ ls -l $SCR/SubSurf-DIGITAL-GEOSX
total 464
drwxr-xr-x 2 mtml g_mtml 4096 Mar 14 14:38 Experimental
```

```

drwxr-xr-x 3 mtml g_mtml 4096 Apr 5 15:19 GEOS
-rwxr-xr-x 1 mtml g_mtml 30123 Jun 13 15:21 GEOS--cache-build-install.sh
-rwxr-xr-x 1 mtml g_mtml 5789 Jun 13 14:15 GEOS-CPU-GCC_10.2.0-mpi--
cache-build-install.sh
-rwxr-xr-x 1 mtml g_mtml 5680 Jun 13 14:07 GEOS-CPU-GCC_10.2.0-mpi--
cache-build-install.sh
-rwxr-xr-- 1 mtml g_mtml 4291 Jun 14 18:24 GEOS_CPU_mpi.sh
-rw-r----- 1 mtml g_mtml 10367 Jun 2 18:19 GEOS-CPU_mpi.slurm
-rwxr-xr-- 1 mtml g_mtml 3946 Jun 14 18:08 GEOS_CPU_omp.sh
-rw-r--r-- 1 mtml g_mtml 10088 Jun 2 18:17 GEOS-CPU_omp.slurm
-rwxr-xr-x 1 mtml g_mtml 5782 Jun 13 14:07 GEOS-GPU-GCC_10.2.0-mpi--
cache-build-install.sh
-rwxr-xr-x 1 mtml g_mtml 5946 Jun 13 14:16 GEOS-GPU-GCC_10.2.0-mpi--
cache-build-install.sh
-rwxr-xr-x 1 mtml g_mtml 4706 Jun 14 09:31 GEOS-quick-help.sh
-rwxr-xr-x 1 mtml g_mtml 4239 May 30 14:55 map_ranks_gpus.sh
-rwxr-xr-x 1 mtml g_mtml 1317 Feb 14 12:17 mkmodulefiles.sh
drwxr-xr-x 2 mtml g_mtml 4096 Jun 16 11:00 modulefiles
-rw-r--r-- 1 mtml g_mtml 43656 Jun 16 11:00 README.md
drwxr-xr-x 4 mtml g_mtml 4096 Apr 5 11:15 Version_1.00

```

Existing Chevron Public GEOS Installations

There are two GEOS installation currently at Chevron. Namely

1. The Chevron "official" one under ***/chap/geos/0.2.0/x86_64/RHEL7/install-host-config-build_type***

```

/chap/geos/
├── 0.2.0
│   ├── x86_64
│   │   ├── RHEL7
│   │   │   ├── install-CPU-OPTO2-Hypre-GCC_10.2.0-mpi_hpcx-OMP-
relwithdebinfo
│   │   │   ├── install-CPU-OPTO3fast-Hypre-GCC_10.2.0-mpi_hpcx-OMP-
relwithdebinfo
│   │   │   ├── install-CPU-OPTO3-Hypre-GCC_10.2.0-mpi_2021.08-OMP-
relwithdebinfo
│   │   │   ├── install-CPU-OPTO3-Hypre-GCC_10.2.0-mpi_hpcx-OMP-
relwithdebinfo
│   │   │   ├── install-GPU-Hypre-GCC-CUDA_11.8-mpi_2021.08-OMP-
relwithdebinfo
│   │   │   ├── install-GPU-Hypre-GCC-CUDA_11.8-mpi_hpcx-OMP-
relwithdebinfo
│   │   └── src
└── modulefiles

```

and,

2. the older public installation under ***/data/saet/software/x86_64/RHEL7/GEOS/0.2.0/install-host-config-build_type***

```

/data/saet/software/x86_64/RHEL7/GEOS/0.2.0
├─ install-CPU-OPT01-Hypre-GCC_10.2.0-mpi_hpcx-relwithdebinfo
├─ install-CPU-OPT02-Hypre-GCC_10.2.0-mpi_hpcx-OMP-relwithdebinfo
├─ install-CPU-OPT02-Hypre-GCC_10.2.0-mpi_hpcx-relwithdebinfo
├─ install-CPU-OPT03fast-Hypre-GCC_10.2.0-mpi_hpcx-OMP-
relwithdebinfo
├─ install-CPU-OPT03fast-Hypre-GCC_10.2.0-mpi_hpcx-relwithdebinfo
├─ install-CPU-OPT03fastmath-Hypre-GCC_10.2.0-mpi_hpcx-OMP-
relwithdebinfo
├─ install-CPU-OPT03-Hypre-GCC_10.2.0-mpi_2021.08-OMP-relwithdebinfo
├─ install-CPU-OPT03-Hypre-GCC_10.2.0-mpi_2021.08-relwithdebinfo
├─ install-CPU-OPT03-Hypre-GCC_10.2.0-mpi_hpcx-OMP-relwithdebinfo
├─ install-CPU-OPT03-Hypre-GCC_10.2.0-mpi_hpcx-relwithdebinfo
├─ install-GPU-Hypre-GCC-CUDA_11.7-mpi_hpcx-OMP-relwithdebinfo
├─ install-GPU-Hypre-GCC-CUDA_11.8-mpi_2021.08-OMP-relwithdebinfo
├─ install-GPU-Hypre-GCC-CUDA_11.8-mpi_2021.08-relwithdebinfo
├─ install-GPU-Hypre-GCC-CUDA_11.8-mpi_hpcx-OMP-relwithdebinfo
└─ install-GPU-Hypre-GCC-CUDA_11.8-mpi_hpcx-relwithdebinfo.

```

This older one will remain accessible and take on the role of GEOS builds with the latest or other experimental features.

Running Models using Existing GEOS Installations

We can immediately run a properly defined GEOS model with any existing public installation as follows:

Look for an "initialization" xxx.rc file under **"/chap/geos/.init"** (or, under **"/data/saet/software/.init/GEOS"**):

```

$ ls -l /chap/geos/.init
Init-x86_64-RHEL7-GEOS-0.2.0-CPU-OPT02-Hypre-GCC_10.2.0-mpi_hpcx-OMP-
relwithdebinfo.rc
Init-x86_64-RHEL7-GEOS-0.2.0-CPU-OPT03fast-Hypre-GCC_10.2.0-mpi_hpcx-
OMP-relwithdebinfo.rc
Init-x86_64-RHEL7-GEOS-0.2.0-CPU-OPT03-Hypre-GCC_10.2.0-mpi_2021.08-
OMP-relwithdebinfo.rc
Init-x86_64-RHEL7-GEOS-0.2.0-CPU-OPT03-Hypre-GCC_10.2.0-mpi_hpcx-OMP-
relwithdebinfo.rc
Init-x86_64-RHEL7-GEOS-0.2.0-GPU-Hypre-GCC-CUDA_11.8-mpi_2021.08-OMP-
relwithdebinfo.rc
Init-x86_64-RHEL7-GEOS-0.2.0-GPU-Hypre-GCC-CUDA_11.8-mpi_hpcx-OMP-
relwithdebinfo.rc

```

These are shell scripts that were generated by our Chevron framework during the build process. They should be used subsequently to prepare the user's environment consistently with the way that GEOS/TPL were built. Pick out one file that fits your needs, such as "Init-x86_64-RHEL7-GEOS-0.2.0-CPU-OPT02-Hypre-GCC_10.2.0-**mpi_hpcx-OMP-relwithdebinfo.rc**". Then to run GEOS in Chevron's SLURM HPC "partition" (batch queue) **rs0**

and account "**rso**", using, say 2 HBv2 nodes (AMD Zen2 architecture, each having 120 cores) with 240 OpenMPI ranks, proceed as follows:

```
ssh toaChevronLinuxBlade

cd /dir/where/your/GEOS/model/is

export INITRC=/chap/geos/.init/Init-x86_64-RHEL7-GEOS-0.2.0-CPU-OPT02-Hypre-
GCC_10.2.0-mpi_hpcx-OMP-relwithdebinfo.rc

V=1 NP=240 PPN=120 sbatch -p rso --account=rso -N 2 --exclusive --constraint=hbv2
-n 240 -c 2 $SCR/GEOS_CPU_omp.sh -i ./<i>myGEOS_model</i>.xml [<i>-x ... -y ...
-z ... other GEOS options</i>]
```

Here is what the above mean

- `GEOS_CPU_omp.sh`: a generic job scripts to be used with OpenMPI GEOS installations and which can be used with any batch system by providing all run-time parameters at the command line; note that there is a similar script `GEOS_CPU_impi.sh` that works with Intel MPI GEOS installations and it can be used in an identical fashion
- `sbatch`: the SLURM job submission command
- `-N`: total number of nodes SLURM should allocate to this job
- `-n`: total number of MPI ranks that will be launched for this job
- `--constraint`: the type of nodes to allocate (hbv2 is an AMD Zen2, hbv3 and AMD Zen3 and hc44 an older Intel Skylake)
- `V`: verbosity to log (0, 1, 2)
- `NP`: total number of MPI ranks to launch
- `PPN`: total number of MPI ranks to launch on each node

Please use `man sbatch` to get the manual page that provides in detail the use of the `sbatch` command. Other commands of interest to SLURM are `salloc` and `sinfo`.

The above example works on the **rso** partition that RPE/RSO teams have access to. Members of Geophysics teams can run GEOS on SLURM partition "**shared**" and by adjusting the `-p` and `--account` options correspondingly. As in

```
V=1 NP=240 PPN=120 sbatch -p shared --account=geophysics -N 2 --exclusive --
constraint=hbv2 -n 240 -c 2 $SCR/GEOS_CPU_omp.sh -i ./<i>myGEOS_model</i>.xml
[<i>-x ... -y ... -z ... other GEOS options</i>]
```

We can check the status of a SLURM job if we know the JobID as follows. The STATE could be "PENDING", that is waiting in the queue to start running, or "RUNNING".

```
$ squeue -l --job 43887
Wed Jun 28 13:35:21 2023
```

	JOBID	PARTITION	NAME	USER	STATE	TIME
TIME_LIMI	NODES	NODELIST (REASON)				
	43887	shared	GEOS_CPU	XXXX	PENDING	0:00
UNLIMITED	2	(Nodes required for job are DOWN, DRAINED or reserved for jobs in higher priority partitions)				

Building GEOS and TPL from Sources

1. The various scripts expect a BASH shell environment but they can be easily used under any shell once we either "jump" to BASH by typing

```
/bin/bash
```

(recommended method), or by prefixing BASH commands/scripts by

```
/bin/bash -c "... bash scripts; bash commands ... "
```

and enclosing entire command lines in double quotes. If you use this method you have to escape embedded ' ' but this can get messy since we often need to also escape the escape character !.

Let's go back to the files and scripts in **SubSurf-DIGITAL-GEOSX-scripts**. Briefly speaking

- GEOS-**CPU**-XXX-**ompi**-XXX.sh (GEOS-**CPU**-XXX-**impi**-XXX.sh) configures, builds and installs CPU-only GEOS for **Open** MPI (Intel MPI), and
- GEOS-**GPU**-XXX-**ompi**-XXX.sh (GEOS-**GPU**-XXX-**impi**-XXX.sh) does the same for GPU-enabled GEOS for **Open** MPI (Intel MPI), respectively.

The *CMAKE* files below request the "OpenMPI HPC-X" MPI stack (**ompi_hpcx**). There are corresponding *CMAKE* files that request the Intel MPI stacks (**impi_2XXX**).

2. Get a "cheat sheet" of the options available to manage the GEOS development workflow, run one of the scripts without any options:

```
$SCR/SubSurf-DIGITAL-GEOSX-scripts/GEOS-CPU-GCC_10.2.0-ompi--cache-build-install.sh
```

```
## GEOS-CPU-GCC_10.2.0-ompi--cache-build-install.sh : clone, configure,
build and install TPL and GEOS with different configurations.
```

```
#
```

```
# Chevron CTC, Innovation and HPC R&D
```

```
#
```

```
## Usage : [optional EnvVar=... See below] ./GEOS-CPU-GCC_10.2.0-ompi--
cache-build-install.sh { CONFIG_NAME | --help }
```

```
##          Locations to clone, install and build GEOS and TPL
```

```
##          [sroot=/full/dir/prefix/to/clone/GEOS];
```

```
default=/home/mtml/src/GEOS_mtml/SubSurf-DIGITAL-GEOSX-scripts
```

```

##          [iroot=/full/dir/prefix/to/install/GEOS];
default=/home/mtml/src/GEOS_mtml/SubSurf-DIGITAL-GEOSX-scripts
##          [croot=/fast/file/system/dir/prefix/to/build/GEOS];
default=/dev/shm/mtml/src; do not modify in not sure of its function
##          [CLONE={0,1}] ; 0 : do not clone, 1 = just clone
##          [BUILD_ONLY={0,1}] ; 0 : clone and not build, 1 = update and
build but do not clone
##          [BUILD_TYPE=] : Release, Debug and RelWithDebInfo
##          [TPL={0,1}] : 1 = perform TPL related tasks, 0 = skip TPL
##          [gitTPL=https://github.com/GEOS-DEV/thirdPartyLibs.git] : change
this to point to another repo, such as Chevron's ADO
##          [TPL_UPDATE=] : 0 = do not update from Github repos; 1 = update
##          [TPL_RESET=] : '--type { HEAD~n | hash }' to reset before
building TPL targets'
##          [TPL_BUILD_ONLY={0|' list '}] : 0 = configure and build all TPL
packages, 'list ' = only rebuild those in 'list'
##          [GEOS={0,1}] : 1 = carry out GEOS related tasks, 0 = skip GEOS
##          [gitGEOS=https://github.com/GEOS-DEV/GEOS.git] : change this to
point to another repo, such as Chevron's ADO
##          [BRANCH=branch_name] : checkout this branch before building GEOS
##          [GEOS_REBUILD={0,1}] : 1 rebuild GEOS without update and
configure steps, 0 = update, configure and build all GEOS targets
##          [GEOS_UPDATE=] : 0 = do not update from Github repos; 1 = update
##          [GEOS_REBASE=] : 0 = apply Git merge, 1 = apply Git merge with
'--rebase' option
##          [GEOS_BUILD_ONLY={0|' list '}] : 0 = configure and build all
targets, 'list ' = only build those in 'list'
##          [GEOS_RESET=] : '--type { HEAD~n | hash }' to reset before
building GEOS targets'
##          [GEOS_INSTALL={0,1}] : 1 = install GEOS targets after build, 0 =
do not install
##          [LCACHE={0,1,2,3,4}] : 0 = do not use local cache, 1 = use, 2 =
clear cache, 3 = copy GEOS build back to sroot
##          [XML_TOOLS_BUILD={0,1}] : 0 = do not build XML tools, 1 = build
##          [ENABLE_CALIPER_HYPRE={OFF,ON}] : OFF = do not build Hype with
Caliper instrumentation, ON = build with
##
##          Compiler tool-chains and MPI stacks
##          [comp=gcc/10.4.0] : set comp to point to a modulefile of the
desired compiler version
##          [mpi=hpcx] : set mpi to point to 'modulefile(s)' of the desired
MPI version
##          [modules=/dev1/geophys/util/modules/ModuleFiles/git/2.27.0
CMake_3.24.1 ] : set modules to point to 'list of modulefiles to also load
##          [CTEST={0,1}] : 1 = run all quality tests on GEOS after
installation
##
##          [V=0] : 0 = minimal, 1 = some, 2 = more verbosity
##          MODULES=[ hpcx /dev1/geophys/util/modules/ModuleFiles/git/2.27.0
CMake_3.24.1 gcc/10.4.0 ] : final set of modulefiles to load before
compiling TPL and GEOS
##
##          HOST_CONFIG_DIR=/home/mtml/src/GEOS_mtml/SubSurf-DIGITAL-GEOSX-
scripts/GEOS/host-configs/CVX directory path prefix for Chevron's host-

```



```

configs; can be changed
##          CONFIG_NAME is this part of the path:
/home/mtml/src/GEOS_mtml/SubSurf-DIGITAL-GEOSX-scripts/GEOS/host-
configs/CVX/CONFIG_NAME.cmake
##
## Quick Instructions for BASH shell. Or type '/bin/bash' first to jump to
BASH
## [] : optional, { | } alternatives
export SCR=/where/you/cloned/SubSurf-DIGITAL-GEOSX-scripts
export sroot=/where/you/want/GEOS/cloned/and/built
export iroot=/where/you/want/GEOS/installed
## First, clone in a clean directory (first time)
{ CLONE=1 | BUILD_ONLY=0 } TPL=1 GEOS=1 ${SCR}/GEOS-CPU-GCC_10.2.0-mpi-
-cache-build-install.sh CONFIG_NAME
## To configure + build + install TPL and GEOS at a particular GEOS BRANCH
or Github commit point:
  BRANCH="CVX/residual-flash" V=1 BUILD_ONLY=1 TPL=1 GEOS=1 ${SCR}/GEOS-
CPU-GCC_10.2.0-mpi--cache-build-install.sh CONFIG_NAME
## To update + configure + build and install both TPL and GEOS afterwards:
  BUILD_ONLY=1 TPL=1 GEOS=1 ${SCR}/GEOS-CPU-GCC_10.2.0-mpi--cache-build-
install.sh CONFIG_NAME
## To update + configure + build and install only GEOS afterwards. No need
to rebuild TPL unless it has been modified:
  BUILD_ONLY=1 GEOS=1 ${SCR}/GEOS-CPU-GCC_10.2.0-mpi--cache-build-
install.sh CONFIG_NAME
## To update + re-build and install only specific TPL packages in 'list' :
  BUILD_ONLY=1 TPL=1 TPL_BUILD_ONLY=' X Y Z ' ${SCR}/GEOS-CPU-GCC_10.2.0-
mpi--cache-build-install.sh CONFIG_NAME
## To update + re-build and install only specific GEOS packages in 'list'
or just geosx :
  BUILD_ONLY=1 GEOS=1 GEOS_BUILD_ONLY={ 1 | geosx } ${SCR}/GEOS-CPU-
GCC_10.2.0-mpi--cache-build-install.sh CONFIG_NAME
## To only re-build and install geosx binary without updating or re-
configuring:
  BUILD_ONLY=1 GEOS=1 GEOS_REBUILD=1 ${SCR}/GEOS-CPU-GCC_10.2.0-mpi--
cache-build-install.sh CONFIG_NAME
## To only re-build but not re-configure after local modifications in GEOS
:
  BUILD_ONLY=1 GEOS=1 GEOS_REBUILD=1 GEOS_BUILD_ONLY=1 ${SCR}/GEOS-CPU-
GCC_10.2.0-mpi--cache-build-install.sh CONFIG_NAME
## To configure + build in sroot without fast local caching and **not
install GEOS** :
  BUILD_ONLY=1 LCACHE=0 GEOS=1 GEOS_INSTALL=0 ${SCR}/GEOS-CPU-GCC_10.2.0-
mpi--cache-build-install.sh CONFIG_NAME
## To configure + build + install and run all unit tests afterwards:
  BUILD_ONLY=1 TPL=1 GEOS=1 CTEST=1 ${SCR}/GEOS-CPU-GCC_10.2.0-mpi--
cache-build-install.sh CONFIG_NAME
##
## Existing CVX CONFIG_NAME.cmake at /home/mtml/src/GEOS_mtml/SubSurf-
DIGITAL-GEOSX-scripts/GEOS/host-configs/CVX; set CVX_CONFIG_DIR to other
location if desired
      CPU-OPTO2-Hypr-GCC_10.2.0-mpi_hpcx-OMP
(/home/mtml/src/GEOS_mtml/SubSurf-DIGITAL-GEOSX-scripts/GEOS/host-
configs/CVX/CPU-OPTO2-Hypr-GCC_10.2.0-mpi_hpcx-OMP.cmake)

```

```

        CPU-OPTO3fast-Hypre-GCC_10.2.0-mpi_hpcx-OMP
(/home/mtml/src/GEOS_mtml/SubSurf-DIGITAL-GEOSX-scripts/GEOS/host-
configs/CVX/CPU-OPTO3fast-Hypre-GCC_10.2.0-mpi_hpcx-OMP.cmake)
        CPU-OPTO3fastmath-Hypre-GCC_10.2.0-mpi_hpcx-OMP
(/home/mtml/src/GEOS_mtml/SubSurf-DIGITAL-GEOSX-scripts/GEOS/host-
configs/CVX/CPU-OPTO3fastmath-Hypre-GCC_10.2.0-mpi_hpcx-OMP.cmake)
        CPU-OPTO3-Hypre-GCC_10.2.0-mpi_hpcx-OMP
(/home/mtml/src/GEOS_mtml/SubSurf-DIGITAL-GEOSX-scripts/GEOS/host-
configs/CVX/CPU-OPTO3-Hypre-GCC_10.2.0-mpi_hpcx-OMP.cmake)

```

3. Select a Linux files system with sufficient free space to build and install TPL and GEOS. We recommend a directory under '/data/...'. Let's say '/data/mine/GEOS'
4. Run the clone-configure-build scripts for Intel MPI or Open MPI with proper parameters to first clone and then configure and build TPL and GEOS under a particular configuration. For instance, to built and install TPL+GEOS in "Release" mode for OpenMPI, do the following

```

$ sroot=/data/mine/GEOS iroot=/data/mine/GEOS \
  CLONE=1 TPL=1 GEOS=1 \
  $SCR/SubSurf-DIGITAL-GEOSX-scripts/GEOS-CPU-GCC_10.2.0-mpi--cache-build-
install.sh \
  CPU-OPTO2-Hypre-GCC_10.2.0-mpi_hpcx

$ sroot=/data/mine/GEOS iroot=/data/mine/GEOS/install \
  BUILD_ONLY=1 TPL=1 GEOS=1 \
  $SCR/SubSurf-DIGITAL-GEOSX-scripts/GEOS-CPU-GCC_10.2.0-mpi--cache-build-
install.sh \
  CPU-OPTO2-Hypre-GCC_10.2.0-mpi_hpcx

```

To built and install TPL+GEOS in "RelWithDebInfo" mode for IntelMPI and OpenMP enabled, do the following

```

$ sroot=/data/mine/GEOS iroot=/data/mine/GEOS \
  CLONE=1 TPL=1 GEOS=1 \
  $SCR/SubSurf-DIGITAL-GEOSX-scripts/GEOS-CPU-GCC_10.2.0-mpi--cache-build-
install.sh \
  CPU-OPTO2-Hypre-GCC_10.2.0-mpi_2021.08-OMP

$ sroot=/data/mine/GEOS iroot=/data/mine/GEOS/install \
  BUILD_ONLY=1 TPL=1 GEOS=1 BUILD_TYPE=RelWithDebInfo \
  $SCR/SubSurf-DIGITAL-GEOSX-scripts/GEOS-CPU-GCC_10.2.0-mpi--cache-build-
install.sh \
  CPU-OPTO2-Hypre-GCC_10.2.0-mpi_2021.08-OMP

```

The first command line clones both TPL and GEOS from GitHub to local directory '/data/mine/GEOS'. The second command builds and installs TPL and then GEOS in a path under '/data/mine/GEOS/install'.

By default the framework builds the 'Release' type of TPL and GEOS. We may request different build types, including,

- **Release** : optimizations applied but no symbolic information for debugging,
- **Debug** : no optimizations applied and symbolic information for debugging included, and
- **RelWithDebInfo** : optimizations applied and symbolic information for debugging included.

We can select the build type by setting the BUILD_TYPE environment variable, as in

```
$ sroot=/data/mine/GEOS iroot=/data/mine/GEOS/install \
  BUILD_ONLY=1 GEOS=1 BUILD_TYPE=RelWithDebInfo \
  $SCR/SubSurf-DIGITAL-GEOSX-scripts/GEOS-CPU-GCC_10.2.0-mpi--cache-build-
install.sh \
  CPU-OPT02-Hypre-GCC_10.2.0-mpi_hpcx
```

If one wants to modify and then test and run the new GEOS code, they should proceed as follows. First, follow the two steps shown above to create initial installations for both TPL and GEOS under a particular configuration. Then we strongly recommend to create a new branch for GEOS and use it to contain all local modifications. Otherwise, the local GEOS clone may become impossible to update with newer changes in the GitHub repos.

When one is ready to test the changes, proceed as follows, to only re-build GEOS :

```
$ sroot=/data/mine/GEOS iroot=/data/mine/GEOS/install \
  BUILD_ONLY=1 GEOS=1 GEOS_REBUILD=1 \
  $SCR/SubSurf-DIGITAL-GEOSX-scripts/GEOS-CPU-GCC_10.2.0-mpi--cache-build-
install.sh \
  CPU-OPT02-Hypre-GCC_10.2.0-mpi_hpcx
```

To configure, build and install TPL and GEOS at a particular GEOS BRANCH or Github commit point: "

```
$ sroot=/data/mine/GEOS iroot=/data/mine/GEOS/install \
  BRANCH="CVX/residual-flash" V=1 BUILD_ONLY=1 TPL=1 GEOS=1 \
  $SCR/SubSurf-DIGITAL-GEOSX-scripts/GEOS-CPU-GCC_10.2.0-mpi--cache-build-
install.sh \
  CPU-OPT02-Hypre-GCC_10.2.0-mpi_hpcx
```

To run GEOS, you need to first initialize your shell with the proper settings. To do so source a matching initialization file that can be found either under your home : '\${HOME}/.init/GEOS' (if you build GEOS locally) or under the public GEOS installations in '/data/saet/software/.init/GEOS'. To use the above example case, the initialization file would be 'Init-x86_64-RHEL7-GEOS-0.2.0-CPU-OPT02-Hypre-GCC_10.2.0-mpi_hpcx-OMP-release.rc'. That is :

```

$ source /data/saet/software/.init/GEOS/Init-x86_64-RHEL7-GEOS-0.2.0-CPU-
OPTO2-Hypre-GCC_10.2.0-mpi_hpcx-release.rc
$ geosx
GEOSX version: 0.2.0 (develop, sha1: cd6d1383d8)
  - c++ compiler: gcc 10.2.0
  - MPI version: Open MPI v4.1.5a1, package: Open MPI root@hpc-kernel-03
Distribution, ident: 4.1.5a1, repo rev: v4.1.4-32-g5abd86cc8c, Unreleased
developer copy
  - HDF5 version: 1.12.1
  - Conduit version: 0.8.2
  - VTK version: 9.1.0
  - RAJA version: 2022.3.0
  - umpire version: 2022.3.0
  - adiak version: ..
  - caliper version: 2.8.0
  - METIS version: 5.1.0
  - PARAMETIS version: 4.0.3
  - scotch version: 6.0.9
  - superlu_dist version: 6.3.0
  - suitesparse version: 5.7.9
  - hypre development version: v2.27.0-9-g52802b646 (master)
USAGE: geosx -i input.xml [options]

```

Options:

```

-?, --help
-i, --input,          Input xml filename (required)
-r, --restart,        Target restart filename
-x, --x-partitions,   Number of partitions in the x-direction
-y, --y-partitions,   Number of partitions in the y-direction
-z, --z-partitions,   Number of partitions in the z-direction
-s, --schema,         Name of the output schema
-b, --use-nonblocking, Use non-blocking MPI communication
-n, --name,           Name of the problem, used for output
-s, --suppress-pinned, Suppress usage of pinned memory for MPI
communication buffers
-o, --output,         Directory to put the output files
-t, --timers,         String specifying the type of timer output
--trace-data-migration, Trace host-device data migration
--pause-for,         Pause geosx for a given number of seconds before
starting execution
...

```

GEOS is MPI code so we have to provide the proper options at the "mpirun" command line. These depend on the MPI stack we use. For instance, here is how we would run GEOS built with GCC and OpenMPI with the "compositionalMultiphaseFlow" example test case that comes with GEOS.

First, we make a copy of the input files to a directory that has sufficient space to receive the output results. Then we initialize our SHELL and finally we issue the mpirun command. (I am omitting the '\$' for easy copy and paste.)

```

export OUTDIR="/data/my/geosx/simulations"
mkdir -p $OUTDIR && cd $OUTDIR
rsync -av
/data/saet/mtml/software/x86_64/RHEL7/GEOS/inputFiles/compositionalMultiphaseFlow
.
cd compositionalMultiphaseFlow

source /data/saet/software/.init/GEOS/Init-x86_64-RHEL7-GEOS-0.2.0-CPU-OPT02-
Hypr-GCC_10.2.0-mpi_hpcx-OMP-relwithdebinfo.rc
mpirun -np 10 $GEOSX_DIR/bin/geosx -i ./4comp_2ph_1d.xml -x 10 -y 1 -z 1

Max threads: 1
GEOSX version: 0.2.0 (develop, sha1: d2cba38af)
- c++ compiler: gcc 10.2.0
- openmp version: 201511
- MPI version: Open MPI v4.1.5a1, package: Open MPI root@hpc-kernel-03
Distribution, ident: 4.1.5a1, repo rev: v4.1.4-32-g5abd86cc8c,
...
- hypr development version: v2.27.0-9-g52802b646 (master)
Adding Solver of type CompositionalMultiphaseFVM, named compflow
Adding Mesh: InternalMesh, mesh1
...
Adding Object CellElementRegion named Region1 from ObjectManager::Catalog.
mesh1: total number of nodes = 44
mesh1: total number of elems = 10
regionQuadrature: meshBodyName, meshLevelName, regionName, subRegionName =
mesh1, Level0, Region1, block1
mesh1/Level0/Region1/block1/fluid1 allocated 1 quadrature points
mesh1/Level0/Region1/block1/rock allocated 1 quadrature points
mesh1/Level0/Region1/block1/relperm allocated 1 quadrature points
mesh1: importing field data from mesh dataset
Time: 0s, dt:1000s, Cycle: 0
Attempt: 0, ConfigurationIter: 0, NewtonIter: 0
( Rflow ) = ( 5.36e-01 ) ; ( R ) = ( 5.36e-01 ) ;
Attempt: 0, ConfigurationIter: 0, NewtonIter: 1
( Rflow ) = ( 4.19e-01 ) ; ( R ) = ( 4.19e-01 ) ;
Last LinSolve(iter,res) = ( 1, 5.25e-19 ) ;
...
( Rflow ) = ( 5.01e-03 ) ; ( R ) = ( 5.01e-03 ) ;
Last LinSolve(iter,res) = ( 1, 5.72e-18 ) ;
Attempt: 0, ConfigurationIter: 0, NewtonIter: 7
( Rflow ) = ( 6.31e-07 ) ; ( R ) = ( 6.31e-07 ) ;
Last LinSolve(iter,res) = ( 1, 3.01e-18 ) ;
compflow: Max relative pressure change: 100 %
compflow: Max absolute phase volume fraction change: 0.871622
compflow: Time-step required will be decreased based on state change.
Rank 0: Writing out restart file at
./4comp_2ph_1d_restart_000000000/rank_0000000.hdf5
...
Rank 4: Writing out restart file at
./4comp_2ph_1d_restart_000000000/rank_0000004.hdf5
Time: 1000s, dt:1000s, Cycle: 1
Attempt: 0, ConfigurationIter: 0, NewtonIter: 0

```

```

( Rflow ) = ( 6.99e-03 ) ;      ( R ) = ( 6.99e-03 ) ;
Attempt: 0, ConfigurationIter: 0, NewtonIter: 1
( Rflow ) = ( 1.93e-05 ) ;      ( R ) = ( 1.93e-05 ) ;
Last LinSolve(iter,res) = ( 1, 3.23e-17 ) ;
Attempt: 0, ConfigurationIter: 0, NewtonIter: 2
( Rflow ) = ( 2.21e-09 ) ;      ( R ) = ( 2.21e-09 ) ;
Last LinSolve(iter,res) = ( 1, 5.76e-16 ) ;
compflow: Max relative pressure change: 1.20537 %
compflow: Max absolute phase volume fraction change: 0.00211988
compflow: Newton solver converged in less than 6 iterations, time-step required
will be increased.
Time: 2000s, dt:1000s, Cycle: 2
Attempt: 0, ConfigurationIter: 0, NewtonIter: 0
( Rflow ) = ( 6.83e-03 ) ;      ( R ) = ( 6.83e-03 ) ;
Attempt: 0, ConfigurationIter: 0, NewtonIter: 1
( Rflow ) = ( 1.83e-05 ) ;      ( R ) = ( 1.83e-05 ) ;
Last LinSolve(iter,res) = ( 1, 8.12e-17 ) ;
Attempt: 0, ConfigurationIter: 0, NewtonIter: 2
...
Rank 3: Writing out restart file at
./4comp_2ph_1d_restart_000000218/rank_0000003.hdf5
Umpire          HOST sum across ranks: 563.4 KB
Umpire          HOST          rank max: 60.3 KB
total time              56.002s
initialization time      0.168s
run time                 51.802s

```

To run these models on an HPC cluster we will have to embed these commands in batch job scripts. Two example batch scripts for SLURM have been included in the SubSurf-DIGITAL-GEOSX-scripts repos, one for Intel and another for Open MPI.

- Support for XML Tools for GEOS GEOS provides a number of "convenient" XML tools that need to be build alongside GEOS itself. We have enabled experimental support for XML tools by setting EnvVar XML_TOOLS_BUILD to a value > 0, as follows:

```

$ sroot=/data/mine/GEOS iroot=/data/mine/GEOS/install \
  BUILD_ONLY=1 GEOS=1 XML_TOOLS_BUILD=1 BUILD_TYPE=RelWithDebInfo \
  $SCR/SubSurf-DIGITAL-GEOSX-scripts/GEOS-CPU-GCC_10.2.0-mpi--cache-build-
install.sh \
  CPU-OPT02-Hypre-GCC_10.2.0-mpi_hpcx

```

As I mentioned support is experimental and has not been thoroughly tested. These tools become accessible after we source the Init-XXXX.rc file that is related to specific GEOS built. Such as

```

$ source ~/.init/GEOS/Init-x86_64-RHEL7-GEOS-0.2.0-CPU-OPT03-Hypre-GCC_10.2.0-
impi_2018.04-OMP-release.rc
...
$ which format_xml

```

```
~/local/bin/format_xml

$ format_xml -h
usage: format_xml [-h] [-i INDENT] [-s STYLE] [-d DEPTH] [-a ALPHEBITIZE] [-c
CLOSE] [-n NAMESPACE] input

positional arguments:
  input                Input file name

optional arguments:
  -h, --help            show this help message and exit
  -i INDENT, --indent INDENT
                        Indent size
  -s STYLE, --style STYLE
                        Indent style
  -d DEPTH, --depth DEPTH
                        Block separation depth
  -a ALPHEBITIZE, --alphebitize ALPHEBITIZE
                        Alphebetize attributes
  -c CLOSE, --close CLOSE
                        Close tag style
  -n NAMESPACE, --namespace NAMESPACE
                        Include namespace
```

5. Updating GEOS Sources

Everytime we run the scripts with TPL=1 and GEOS=1, the local repos receive the latest updates from GitHub. If we just want to update GEOS's sources but build it at a later time, simply invoke the scripts with sroot and any host-configs CMAKE file, as in

```
$ sroot=/data/mine/GEOS $SCR/SubSurf-DIGITAL-GEOSX-scripts/GEOS-CPU-
GCC_10.2.0-mpi--cache-build-install.sh \
CPU-OPTO2-Hypre-GCC_10.2.0-mpi_hpcx
```

This is useful if we need to update GEOS sources from a host that has proper access to the Internet resources and build it on a faster host that may not have proper access.

Why Things don't Work as Expected

Many users like to "hard-wire" the settings in their shells to point to specific Python binaries, language compilers, libraries and so on. This "unclean" environment will most likely lead to picking inconsistent, unexpected or undesired packages or versions of these components. It is more likely that the scripts here will fail if these are set already in an inconsistent manner.

Please investigate your "login start up scripts" (~/.bashrc, ~/.bash_profile, ~/.user_opt.env, etc.) and the scripts they source to ensure that the settings initialization is done in a consistent fashion.

We strongly recommend the use of **Environment Modules** [https://chevron.sharepoint.com/_s/HPCHome/Eb_53W5sDzhKlVl16JPbKVABfA4DrbluYXR8ym5wzdcjnQ?e=g7hHb8] that setup and reset the user shell environment in a tidy and consistent fashion.

Running the scripts directly on a cluster node

GEOS is an MPI communications intensive application (like, for instance, Intersect). The best location for properly building GEOS is on a HPC cluster node. Below we show how to do all the interactive compilation and testing directly on a cluster node.

- Get an "interactive" SLURM batch job (e.g., 2 nodes at the RPE cluster; '\$' is the shell prompt) :

```
$ salloc --x11 -N 2 --constraint=hbv3 --account=rso -p rso --exclusive
```

- Upon getting a shell prompt, you can start interacting with the shell directly. "cd" to the location you would like to establish the GEOS area

```
$ mkdir myGEOS; cd myGEOS
```

- Proceed to clone SubSurf-DIGITAL-GEOSX-scripts and run the scripts that build GEOS.
- Follow the instructions at the top under Getting Started

Use of modulefiles and initialization scripts

This framework generates for each GEOS/TPL build a corresponding modulefile and an initialization script. The modulefiles are stored at `${HOME}/modulefiles/GEOS` and at `${iroot}/modulefiles/GEOS`. Initialization scripts are stored at `${HOME}/.init` and `${iroot}/.init` locations.

The files in `.init` should be sourced at the user's environment to create an environment identical to the one used when building GEOS/TPL. This takes care of the proper initialization of the environment modules package for the user. To initialize a specific incarnation of GEOS/TPL we simply source it (BASH shells), e.g.,

```
$ source ${HOME}/.init/GEOS/Init-x86_64-RHEL7-GEOS-0.2.0-install-CPU-  
OPT02-HyPre-GCC_10.2.0-impi_2021.06-release.rc
```

Currently installed GEOS/TPL variants have both public modulefile and `.init` files available, at `/data/saet/software/modulefiles/GEOS` and `/data/saet/software/.init/GEOS`, respectively. We may directly use these can simply


```
$ source /data/saet/software/.init/GEOS/Init-x86_64-RHEL7-GEOS-0.2.0-  
CPU-OPT02-Hypre-GCC_10.2.0-impi_2021.06-release.rc
```

OR

```
$ source /data/saet/hpcrnd/utils/bin/modules_init.sh  
$ module use /data/saet/software/modulefiles  
$ module load GEOS/x86_64-RHEL7-GEOS-0.2.0-CPU-NOOPT-Hypre-GCC_10.2.0-  
impi_2018.04-OMP-release
```

Software dependencies

- The scripts (to all extents possible) setup the user's environment properly and retrieve all latest necessary components from GEOS GitHub "<https://github.com/GEOS-DEV/GEOS.git>"

Latest releases

- This repo contains the latest s/w.

Installation Features and Options

Scripts

The two scripts below completely automate the process of cloning, configuring, building and installing GEOS and thirdPartyLibs to locations that are chosen by the individual developer, as follows

- GEOS-CPU-GCC_10.2.0-impi_2021.06--cache-build-install.sh builds GEOS and TPL using Intel MPI versions OneAPI (2021.06) and 2018.xx, and
- GEOS-CPU-GCC_10.2.0-ompi_hpcx--cache-build-install.sh builds GEOS and TPL using OpenMPI HPC-X (4.1.x).

HPC-X is an accelerated version of stock OpenMPI that is actively developed by engineers at the company that manufactures the Infiniband HPC fabric (was Mellanox, and now Nvidia). We recommend OpenMPI HPC-X if OpenMPI is your choice due to the increased performance and better features.

CVX Host Configuration .cmake files

This repo also distributes a number of CVX-specific

host-config/*.cmake files that support different choices in terms of enabling different MPI stacks, enabling OpenMP and different levels of optimization options. These cmake files are under

./SubSurf-DIGITAL-GEOSX-scripts/GEOS/CVX/host_configs/CVX.

The naming convention of a cmake file reflects compiler, MPI and options choices. For instance

- CPU-OPTO1-Hypre-GCC_10.2.0-impi_2021.06.cmake configures and builds TPL and GEOS for an Intel or Intel-compatible CPU platform, uses linear system solvers from the Hypre package and compiles everything with GCC 10.2.0 and Intel MPI 2021.06 (OneAPI). It requests a -O1 optimization level which is quite mild.
- CPU-OPTO1-Hypre-GCC_10.2.0-ompi_hpcx.cmake applies the same options except that it build everything to use OpenMPI.

Most .cmake files specify -fno-fast-math since a "fast" math option usually lets several of the test codes fail due to floating-point accuracy issues.

Other compilers (e.g., C++LANG) can be added, if needed.

Finally, the scripts take care of initializing the compilation environment with all the prerequisites, including a Python v3, git, git-lfs, the language compilers and MPI.

Overall the scripts clone (the 1st time) and subsequently update directly from the GitHub all TPL and GEOS packages.

The scripts also allow the user to specify where the codes will be cloned, built and installed.

By default the scripts will clone TPL and GEOS at \${sroot}/thirdPartyLibs and \${sroot}/GEOS. The build takes place at directories

- \${sroot}/GEOS/GEOS/build-XXXX for GEOS and
- \${sroot}/GEOS/thirdPartyLibs/build-XXX for TPL.

The installation location is controlled by \${iroot} variable as follows

- \${iroot}/.../GEOSTPL/.../install-CONFIG_NAME-release/... for TPL and
- \${iroot}/.../GEOS/.../install-CONFIG_NAME-release/... for GEOS.

If we specify no \${sroot} and no \${iroot} variables, the cloning, built and installation defaults to the directory in which we run the scripts. and locations to the location where we are running the scripts.

Examples

The scripts expect one argument, the *CONFIG_NAME*, which should match with a configuration .cmake at SubSurf-DIGITAL-GEOSX-scripts/GEOS/chost_configs/CVX/*CONFIG_NAME*.cmake . Do not specify the entire file path, just the matching *CONFIG_NAME*.

For [t]cshell Users

If you are using C-shell or TC-shell, wrap any command line execution as follows

```
/bin/bash -c "command line in BASH syntax"
```

or, simply jump to BASH by

```
/bin/bash
```

Build and Test

Let's assume that we want to download and build TPL and GEOS at `sroot="/data/RPE/maelstrom"` and install it under `iroot="/data/RPE/maelstrom/software"` and we want to use OpenMPI ("HPC-X"), no OpenMP and with `g++` optimization level for C++ set to `"-O2"` we would issue (in the BASH or [K]SH shells) :

- `$ sroot="/data/RPE/maelstrom" iroot="/data/RPE/maelstrom/software" ./GEOS-CPU-GCC_10.2.0-mpi--cache-build-install.sh CPU-OPTO2-Hypr-GCC_10.2.0-mpi_2021.06`

to use a build that enables OpenMP we would issue

- `$ sroot="/data/RPE/maelstrom" iroot="/data/RPE/maelstrom/software" ./GEOS-CPU-GCC_10.2.0-mpi--cache-build-install.sh CPU-OPTO2-Hypr-GCC_10.2.0-mpi_2021.06-mpi_hpcx-OMP`

Please feel free to peruse the various `.cmake` files and understand how to enable different options for your personalized GEOS builds.

First time use

When we run the script for the first time we do the initial cloning. During subsequent we do not want to close again but pull all updates in from "origin" configure and build TPL and GEOS.

To do the initial cloning of both TPL and GEOS we set `BUILD_ONLY=0` and run the scripts as follows :

- `$ CLONE=1 TPL=1 GEOS=1 ./GEOS-CPU-GCC_10.2.0-mpi--cache-build-install.sh CONFIG_NAME` No codes is built during this time.

Subsequent config-build-install on GEOS only

To build TPL and or GEOS afterwards, we issue ('[]' means optional but at least one needs to be given)

- `$ BUILD_ONLY=1 [TPL=1] [GEOS=1] ./GEOS-CPU-GCC_10.2.0-mpi--cache-build-install.sh CONFIG_NAME`

More example use cases

Configure and Build GEOS with Hypr and Intel MPI 2021.08, optimization level `-O2`, no OpenMP, build and install on current directory

- `$ BUILD_ONLY=1 TPL=1 GEOS=1 ./GEOS-CPU-GCC_10.2.0-mpi--cache-build-install.sh CPU-OPTO2-Hypr-GCC_10.2.0-mpi_2021.08`

Configure and Build GEOS with Hypre and Intel MPI 2021.08, optimization level -O2, with OpenMP support, build at /data/XXX/src and install on /data/XXX/bin

- `$ BUILD_ONLY=1 TPL=1 GEOS=1 sroot="/data/XXX/src" iroot="/data/XXX/bin" ./GEOS-CPU-GCC_10.2.0-mpi--cache-build-install.sh CPU-OPT02-Hypre-GCC_10.2.0-mpi_2021.08-OMP`

Configure and Build GEOS with Hypre and Intel MPI 2018.04, optimization level -O2, no OpenMP, build and install on current directory

- `$ BUILD_ONLY=1 TPL=1 GEOS=1 mpi="intel/classic/Intel-mpi_2018.04" ./GEOS-CPU-GCC_10.2.0-mpi--cache-build-install.sh CPU-OPT02-Hypre-GCC_10.2.0-mpi_2018.04`

Configure and Build GEOS with Hypre and OpenMPI hpc-x, optimization level -O3, no OpenMP, build and install on current directory

- `$ BUILD_ONLY=1 TPL=1 GEOS=1 ./GEOS-CPU-GCC_10.2.0-mpi--cache-build-install.sh CPU-OPT03-Hypre-GCC_10.2.0-mpi_hpcx`

Configure and Build GEOS with Hypre and OpenMPI hpc-x, no optimization -O0, no OpenMP, build and install on current directory

- `$ BUILD_ONLY=1 TPL=1 GEOS=1 ./GEOS-CPU-GCC_10.2.0-mpi--cache-build-install.sh CPU-NOOPT00-Hypre-GCC_10.2.0-mpi_hpcx`

Rebuilding just GEOS

TPL packages receive updates much less frequently compared to GEOS code that gets changed a times a day. It is recommended that for each configuration we build TPL **once** and then only rebuild GEOS against it as often as needed to capture changes.

- `$ BUILD_ONLY=1 TPL=1 GEOS=0 ./GEOS-CPU-GCC_10.2.0-mpi--cache-build-install.sh CPU-OPT03-Hypre-GCC_10.2.0-mpi_hpcx`
- `$ BUILD_ONLY=1 TPL=0 GEOS=1 ./GEOS-CPU-GCC_10.2.0-mpi--cache-build-install.sh CPU-OPT03-Hypre-GCC_10.2.0-mpi_hpcx`
- ... update, re-configure and rebuild to capture updates on GEOS
- `$ BUILD_ONLY=1 TPL=0 GEOS=1 ./GEOS-CPU-GCC_10.2.0-mpi--cache-build-install.sh CPU-OPT03-Hypre-GCC_10.2.0-mpi_hpcx` do not update, do not re-configure and just rebuild GEOS after local source code changes
- `$ BUILD_ONLY=1 TPL=0 GEOS_UPDATE=0 GEOS=1 GEOS_REBUILD=1 ./GEOS-CPU-GCC_10.2.0-mpi--cache-build-install.sh CPU-OPT03-Hypre-GCC_10.2.0-mpi_hpcx`

Testing the generated GEOS capabilities

After a successful build/install we may set CTEST=1 envvar to request the script to run a series of unit-tests that exercises correctness and floating-point accuracy aspects of GEOS/TPL. These tests are one of the better ways to ensure that the code will generate accurate results for the models we supply s input. The script will create a directory with the results of the successful and failed tests under `${sroot}/GESOX/build-CONFIG_NAME-Release/Testing`

To run the tests, we simply use

- `$ CTEST=1 TPL=1 GEOS=1 ./GEOS-CPU-GCC_10.2.0-mpi--cache-build-install.sh CONFIG_NAME`

If we do not want to run the tests, we set CTEST=0, as in

- `$ CTEST=0 TPL=1 GEOS=1 ./GEOS-CPU-GCC_10.2.0-mpi--cache-build-install.sh CONFIG_NAME`

Some Troubleshooting

When we build TPL there seem to be an issue with package "scotch" not building in the first pass. This is likely caused by the concurrent TPL package builds with Make building scotch before its prerequisite libraries. To alleviate this issue once we, for example, issue

- `$ sroot=${HOME}/src/GEOS_new BUILD_ONLY=1 TPL=1 GEOS=1 ./GEOS-CPU-GCC_10.2.0-mpi--cache-build-install.sh CPU-OPT02-Hypr-GCC_10.2.0-mpi_2021.08-OMP`

and see something like:

```
...
- HDF5 is parallel:  TRUE
-- Found Conduit:
/data/saet/mtml/software/x86_64/RHEL7/GEOSTPL/0.2.0/install-CPU-OPT02-
Hypr-GCC_10.2.0-mpi_2021.08-OMP-release/conduit (found version 0.8.2)
-- CONDUIT_VERSION          = 0.8.2
-- CONDUIT_INSTALL_PREFIX   =
/data/saet/mtml/software/x86_64/RHEL7/GEOSTPL/0.2.0/install-CPU-OPT02-
Hypr-GCC_10.2.0-mpi_2021.08-OMP-release/conduit
-- CONDUIT_IMPORT_ROOT      =
/data/saet/mtml/software/x86_64/RHEL7/GEOSTPL/0.2.0/install-CPU-OPT02-
Hypr-GCC_10.2.0-mpi_2021.08-OMP-release/conduit
-- CONDUIT_USE_CXX11        = TRUE
-- CONDUIT_USE_FMT          = TRUE
-- CONDUIT_INCLUDE_DIRS     =
/data/saet/mtml/software/x86_64/RHEL7/GEOSTPL/0.2.0/install-CPU-OPT02-
Hypr-GCC_10.2.0-mpi_2021.08-OMP-release/conduit/include/conduit
-- CONDUIT_FORTRAN_ENABLED  = FALSE
-- CONDUIT_PYTHON_ENABLED   =
-- CONDUIT_PYTHON_EXECUTABLE =
-- CONDUIT_PYTHON_MODULE_DIR =
/data/saet/mtml/software/x86_64/RHEL7/GEOSTPL/0.2.0/install-CPU-OPT02-
Hypr-GCC_10.2.0-mpi_2021.08-OMP-release/conduit/python-modules/
-- Conduit Relay features:
-- CONDUIT_RELAY_WEBSERVER_ENABLED = TRUE
-- CONDUIT_RELAY_HDF5_ENABLED      = TRUE
-- CONDUIT_HDF5_DIR                =
/data/saet/mtml/software/x86_64/RHEL7/GEOSTPL/0.2.0/install-CPU-OPT02-
Hypr-GCC_10.2.0-mpi_2021.08-OMP-release/hdf5
-- CONDUIT_RELAY_ADIOS_ENABLED     = FALSE
-- CONDUIT_ADIOS_DIR               =
-- CONDUIT_RELAY_SILO_ENABLED      = FALSE
-- CONDUIT_SILO_DIR                =
```

```

-- CONDUIT_RELAY_MPI_ENABLED          = TRUE
-- Conduit imported targets: conduit::conduit conduit::conduit_mpi
----> Conduit_VERSION = 0.8.2
-- SILO_DIR = /data/saet/mtml/software/x86_64/RHEL7/GEOSTPL/0.2.0/install-
CPU-OPTO2-Hypre-GCC_10.2.0-impi_2021.08-OMP-release/silo
-- PUGIXML_DIR =
/data/saet/mtml/software/x86_64/RHEL7/GEOSTPL/0.2.0/install-CPU-OPTO2-
Hypre-GCC_10.2.0-impi_2021.08-OMP-release/pugixml
----> pugixml_VERSION =
-- RAJA_DIR = /data/saet/mtml/software/x86_64/RHEL7/GEOSTPL/0.2.0/install-
CPU-OPTO2-Hypre-GCC_10.2.0-impi_2021.08-OMP-release/raja
----> RAJA_VERSION=2022.3.0
-- UMPIRE_DIR =
/data/saet/mtml/software/x86_64/RHEL7/GEOSTPL/0.2.0/install-CPU-OPTO2-
Hypre-GCC_10.2.0-impi_2021.08-OMP-release/chai
----> umpire_VERSION=2022.3.0
-- CHAI_DIR = /data/saet/mtml/software/x86_64/RHEL7/GEOSTPL/0.2.0/install-
CPU-OPTO2-Hypre-GCC_10.2.0-impi_2021.08-OMP-release/chai
----> chai_VERSION=
-- ADIAK_DIR = /data/saet/mtml/software/x86_64/RHEL7/GEOSTPL/0.2.0/install-
CPU-OPTO2-Hypre-GCC_10.2.0-impi_2021.08-OMP-release/adiak
----> adiak_VERSION = 0.2.2
-- CALIPER_DIR =
/data/saet/mtml/software/x86_64/RHEL7/GEOSTPL/0.2.0/install-CPU-OPTO2-
Hypre-GCC_10.2.0-impi_2021.08-OMP-release/caliper
----> caliper_VERSION = 2.8.0
-- MATHPRESSO_DIR =
/data/saet/mtml/software/x86_64/RHEL7/GEOSTPL/0.2.0/install-CPU-OPTO2-
Hypre-GCC_10.2.0-impi_2021.08-OMP-release/mathpresso
-- METIS_DIR = /data/saet/mtml/software/x86_64/RHEL7/GEOSTPL/0.2.0/install-
CPU-OPTO2-Hypre-GCC_10.2.0-impi_2021.08-OMP-release/metis
----> METIS_VERSION = 5.1.0
-- PARAMETIS_DIR =
/data/saet/mtml/software/x86_64/RHEL7/GEOSTPL/0.2.0/install-CPU-OPTO2-
Hypre-GCC_10.2.0-impi_2021.08-OMP-release/parametis
----> PARAMETIS_VERSION = 4.0.3
-- SCOTCH_DIR =
/data/saet/mtml/software/x86_64/RHEL7/GEOSTPL/0.2.0/install-CPU-OPTO2-
Hypre-GCC_10.2.0-impi_2021.08-OMP-release/scotch

CMake Error at cmake/thirdparty/SetupGeosxThirdParty.cmake:46 (message):
Could not find 'scotch.h' in
'/data/saet/mtml/software/x86_64/RHEL7/GEOSTPL/0.2.0/install-CPU-OPTO2-
Hypre-GCC_10.2.0-impi_2021.08-OMP-release/scotch/include'

Call Stack (most recent call first):
  cmake/thirdparty/SetupGeosxThirdParty.cmake:470 (find_and_register)
  cmake/CMakeBasics.cmake:49 (include)
  CMakeLists.txt:43 (include)

-- Configuring incomplete, errors occurred!
...

```

we just reissue the same command that failed but with a new envvar setting **TPL_REBUILD=scotch**, as in

```
TPL_REBUILD=scotch sroot=${HOME}/src/GEOS_new VERBOSE=1 BUILD_ONLY=1 TPL=1
GEOS=1 ./GEOS-CPU-GCC_10.2.0-impi_2021.06--cache-build-install.sh CPU-
OPTO2-HyPre-GCC_10.2.0-impi_2021.08-OMP
```

Running the scripts directly on a HPC cluster node through an "Interactive PBS or SLURM Job"

We can run these scripts directly on a node that is identical to where GEOS will be running using an interactive PBS job. This method is quite useful and it may be the only way to build packages when a node has a piece of h/w that the blades lack.

```
[mtml@lxspusc0104 ~] $ cd SubSurf-DIGITAL-GEOSX-scripts
[mtml@lxspusc0104 SubSurf-DIGITAL-GEOSX-scripts]$ qsub -I -X -V -l
select=2:ncpus=120:mpiprocs=1:node=hdr120:platform=amd -lplace=scatter:excl -
qappphou001@az-hdr120
qsub: waiting for job 71144.appphou0001 to start
qsub: job 71144.appphou0001 ready
[mtml@ccnpusc600002p ~]$ cd $PBS_O_WORKDIR
[mtml@ccnpusc600002p SubSurf-DIGITAL-GEOSX-scripts]$ ls -la
total 352
drwx----- 4 mtml 244579 4096 Aug 11 12:19 .
drwx----- 22 mtml 244579 16384 Aug 11 12:15 ..
-rwx----- 1 mtml 244579 136435 Jul 27 11:49 coupled_benchmark_CVX_small.zip
-rwx----- 1 mtml 244579 133729 Jul 27 11:49 coupled_benchmark_CVX.zip
drwx----- 3 mtml 244579 4096 Jul 27 11:51 GEOS
-rwx----- 1 mtml 244579 11993 Aug 10 15:13 GEOS--cache-build-install.sh
-rwx----- 1 mtml 244579 12180 Aug 10 15:13 GEOS-CPU-GCC_10.2.0-mpi--cache-
build-install.sh
-rwx----- 1 mtml 244579 12221 Aug 10 15:13 GEOS-CPU-GCC_10.2.0-mpi--cache-
build-install.sh
drwx----- 9 mtml 244579 4096 Aug 11 12:19 .git
-rw-r--r-- 1 mtml 244579 9094 Aug 11 12:19 README.md
```

Now you are interacting directly with a cluster node and you can go about do any interactive work as before.

Existing (Public) GEOS Installations at Chevron

GEOS (and TPL) has been compiled and installed in **"/data/saet/software/x86_64/RHEL7/GEOS"** (and **"/data/saet/software/x86_64/RHEL7/GEOSTPL**, respectively) with different options and MPI stacks. You may directly run GEOS if you only just need to test out or experiment with different models. The time-stamp on the directories suggests the build date so older builds may be missing more recent code features.

The GEOS build scripts generate initialization scripts (Init-XXXX.rc) that may be sourced in the BASH shell to set up your environment with all the settings that are identical to those of the environment that was active when GEOS/TPL were built. This simplifies the task of reproducing the correct environment prior to running GEOS/TPL that were built with particular MPI stacks and other libraries. These scripts are under `.../init/GEOS/` directories that can be found in a number of different places:

- `/data/saet/software/.init/GEOS/`
- `${HOME}/.init/GEOS/` if you built GEOS, or even
- `/data/saet/software/x86_64/RHEL7/GEOS/0.2.0/*/init/`, under each particular GEOS build.

To use any of the existing installations, follow these steps in your interactive SHELL or in our batch scripts.

1. Switch to BASH (if your login SHELL is not BASH) :

```
$ /bin/bash
```

2. Source the initialization script associated with the particular installation. If you would like to use, say CPU-OPTO2-Hypr-GCC_10.2.0-mpi_hpcx-OMP-relwithdebinf (GEOS built with decent optimization options, OMP enabled and OpenMPI)

```
$ source /data/saet/software/.init/GEOS/Init-x86_64-RHEL7-GEOS-0.2.0-CPU-
OPTO2-Hypr-GCC_10.2.0-mpi_hpcx-OMP-relwithdebinf.rc
```

We are updating in a non-systematic fashion the installations of GEOS. Currently, these are already installed

```
/data/saet/software/x86_64/RHEL7/GEOS
└─ 0.2.0
    ├── install-CPU-OPTO1-Hypr-GCC_10.2.0-mpi_hpcx-OMP-relwithdebinf
    ├── install-CPU-OPTO1-Hypr-GCC_10.2.0-mpi_hpcx-relwithdebinf
    ├── install-CPU-OPTO2-Hypr-GCC_10.2.0-mpi_hpcx-OMP-relwithdebinf
    ├── install-CPU-OPTO2-Hypr-GCC_10.2.0-mpi_hpcx-relwithdebinf
    ├── install-CPU-OPTO3fast-Hypr-GCC_10.2.0-mpi_hpcx-relwithdebinf
    ├── install-CPU-OPTO3-Hypr-GCC_10.2.0-mpi_2021.08-OMP-relwithdebinf
    ├── install-CPU-OPTO3-Hypr-GCC_10.2.0-mpi_2021.08-relwithdebinf
    ├── install-CPU-OPTO3-Hypr-GCC_10.2.0-mpi_hpcx-OMP-relwithdebinf
    ├── install-CPU-OPTO3-Hypr-GCC_10.2.0-mpi_hpcx-relwithdebinf
    ├── install-GPU-Hypr-GCC-CUDA_11.8-mpi_2021.08-OMP-relwithdebinf
    ├── install-GPU-Hypr-GCC-CUDA_11.8-mpi_2021.08-relwithdebinf
    ├── install-GPU-Hypr-GCC-CUDA_11.8-mpi_hpcx-OMP-relwithdebinf
    └─ install-GPU-Hypr-GCC-CUDA_11.8-mpi_hpcx-relwithdebinf
```

To run existing GEOS installations directly you will have to initialize your environment properly using "Environment Modules". We are strongly recommending to do this in an Interactive PBS batch job's environment as we've shown previously.

Using TPL Packages Independently

If you would like to use the TPL libraries GEOS uses as prerequisites to build other packages, look at `"/data/saet/software/x86_64/RHEL7/GEOSTPL"`. All packages there have been built with options corresponding to the GEOS built.

Please contact us if you need clarifications and assistance.

Contribute

I would like to keep this repo consistent and allow for modifications using Pull Requests. Please avoid "pushing" back to this repo your local modifications.

Issue a Pull Request (PR) to notify us of changes that you would like to incorporate in this repo.

If you want to learn more about creating good readme files then refer the following [guidelines](#). You can also seek inspiration from the below readme files:

- [Visual Studio Code](#)